

A. Opcode Sequences Extraction

To ensure the reproducibility of our results, this section provides a more detailed description of the opcode sequences extraction pipeline, corresponding to our implementation. The core principle is to isolate and extract logic exclusively from user-defined functions, filtering out compiler-generated artifacts and standard library code:

- **Native-Compiled Languages (C/C++):** Our pipeline first identifies user-defined functions directly from the source code. After compilation, it disassembles the binary but only extracts instruction sequences from these pre-identified functions. This source-aware approach effectively filters out opcode sequences from linked libraries and compiler-generated helper functions.
- **Managed-Runtime Languages (Java/C#):** To process code snippets, we employ a code-wrapping strategy that places the user’s logic within a standard, compilable class structure. This creates a well-defined scope, allowing our tools to extract instruction sequences exclusively from the user-provided code after compilation, ignoring other boilerplate.
- **Interpreted Languages (Python):** The process is more straightforward, utilizing the language’s built-in introspection tools. This directly disassembles the source string, naturally limiting the scope of opcode extraction to the user’s code and avoiding external or interpreter-level noise.

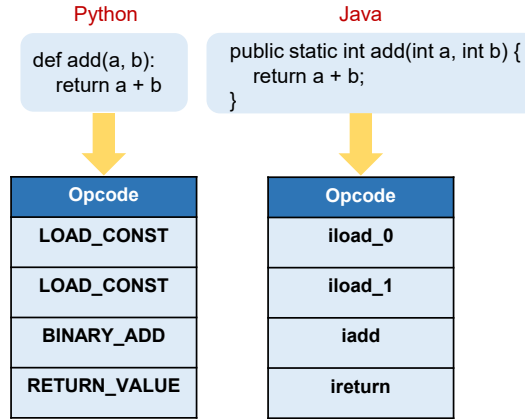


Figure 1: An illustration of converting a simple add function into its corresponding opcode sequences in Python and Java.

This meticulous, source-aware extraction strategy is crucial. For each program, it yields a sequence of low-level computational operations specific to its language’s execution environment. Although the vocabularies of these sequences differ across languages, they provide a more direct representation of the underlying functional logic than source code, creating a robust foundation for our model to subsequently learn a shared, language-agnostic semantic space.

To intuitively demonstrate how opcode sequences strip away syntactic differences and reveal the underlying functional logic, we present an example of a simple add function in Figure 1. As illustrated, although the Python and Java source codes are syntactically distinct, their resulting opcode sequences show a high degree of consistency in their functional steps: both perform the core operations of loading two operands, executing an addition, and returning the result. This clearly demonstrates that opcode sequences, as a cross-language intermediate representation closer to the program’s core execution logic, can effectively help our model capture language-agnostic functional semantics.

Token Range	#Problems	Python	Java	C	C#	Avg.Tokens	Total
(0, 256]	407	2,035	1,992	1,978	1,315	150	7,320
(256, 512]	436	1,874	2,081	1,507	1,370	353	6,832
(512, 1024]	253	933	1,115	543	708	688	3,299
(1024, ∞)	92	263	422	171	317	2215	1,173
Total	1,188	5,105	5,610	4,199	3,710	852	18,624

Table 1: CodeNet-VL Dataset Statistics

B. CodeNet-VL Dataset Construction Process

In order to comprehensively evaluate our model, especially its performance in handling codes of different lengths, we constructed a new benchmark dataset called CodeNet-VL (Variable-Length), which is based on the large-scale CodeNet dataset.

Construction of CodeNet-VL dataset

The construction of CodeNet-VL follows a rigorous data processing process. First, to ensure the functional correctness and logical integrity of the samples, we only select code submissions with an evaluation status of Accepted from CodeNet as the data source. On this basis, we define functional equivalence as the ability to solve the same problem ID, that is, all Accepted submissions from the same problem are regarded as functional semantic clones. Subsequently, in order to stratify the code length, we use the CodeBERT tokenizer consistent with the model input to tokenize each code file and accurately count the number of tokens. Finally, we constructed the CodeNet-VL dataset by stratified random sampling from the functional equivalence pool. The dataset is divided into four length intervals, covering various scenarios from short code to very long code. Short codes (0, 512] are usually implementations of basic algorithms or simple functions, which are within the input limit of standard pre-trained models (PLMs). Long codes (512, 1024] and ultra-long codes (1024, ∞) are beyond the processing range of the standard PLM, posing a challenge to the model’s ability to process long sequence information. Detailed statistics are shown in Table 1.

C. Hyperparameter sensitivity analysis

To verify the suitability of key hyperparameters in our model, we conducted a series of sensitivity analysis experiments. Specifically, the training-related hyperparameters were tuned on the validation set of the CSN_{CC} dataset, and the reported MAP values were the average of the two retrieval directions of Python→Java and Java→Python. The hyperparameters related to the long code reasoning strategy were verified on the (1024, ∞) interval of the CodeNet-VL dataset, and its MAP value was also the average result of the Python→Java and Java→Python directions.

LC3-Dual Training Hyperparameters

Auxiliary loss weights α and β are the weights of L_{align} (adversarial alignment loss) and L_{cycle} (cycle consistency loss), respectively. They are used to balance the contributions of different objectives in multi-task learning. To find the best balance between these two weight factors, we use a grid search method to test combinations of α and β in a predefined parameter space of 0.5, 1.0. The experimental results are shown in Table 2. The grid search results clearly show that the model performance is

$\alpha \setminus \beta$	0.5	1.0
0.5	64.23	65.72
1.0	64.67	65.96

Table 2: Grid search results for loss weights α and β (MAP %)

best when α and β are both set to 1.0. This shows that giving the two auxiliary tasks the same weight as the main contrastive loss can most effectively guide the model to learn language-independent and structurally consistent representations. Therefore, we choose $\alpha=1.0$, $\beta=1.0$ as the final configuration.

Operation Sequence Encoder Selection

In order to select the most effective encoder for the operation sequence, we compared three widely used sequence models: Gated Recurrent Unit (GRU), Transformer, and Bidirectional Long Short-Term Memory (BiLSTM). We conducted comparative experiments on the CSN_{CC} dataset, and the results are shown in Table 3. Experimental results show that BiLSTM significantly

Method	MAP
BiLSTM	65.96
Transformer	62.59
GRU	61.03

Table 3: Performance comparison of different Opcode encoders (%)

outperforms Transformer and GRU in all evaluation metrics. Although the Transformer architecture theoretically has an advantage in capturing long-distance dependencies due to its self-attention mechanism, BiLSTM performs better in our specific task. We speculate that this may be due to several factors: first, the opcode sequences have a strong local order, and the recurrent structure of BiLSTM is naturally suitable for capturing this sequence dependency; second, compared to the Transformer, which requires large amounts of data to fully realize its potential, BiLSTM may be easier to train and achieve better convergence given our dataset size. In addition, as a relatively lightweight model, BiLSTM achieves the best balance between computational overhead and performance. Therefore, we choose BiLSTM as the encoder of the opcode sequences in our final model.

Semantic Affinity Aggregation Hyperparameters

Sliding Window Stride When chunking long codes, we fixed the window size to 512 tokens and explored its impact on model performance by adjusting the overlap between consecutive windows. The stride is determined by the overlap size, following the equation: $\text{Stride} = 512 - \text{Overlap}$. We tested the performance under different overlap settings, and the results are shown in Table 4. Table 4 shows that moderate overlap is crucial for capturing contextual information across block boundaries.

Overlap	Stride	MAP
0	512	59.06
64	448	62.78
128	384	65.43
192	320	65.46
256	256	63.16

Table 4: Sensitivity analysis of sliding window overlap(%)

When the overlap increases from 0 to 128 tokens, the MAP performance of the model is significantly improved. Although the performance reaches its highest point at 192 tokens, the improvement compared to 128 tokens is extremely limited. Considering that a larger overlap will incur higher computational costs, we believe that an overlap of 128 tokens is a better trade-off between performance gains and computational efficiency. Therefore, we choose an overlap of 128 tokens, that is, a step size of 384 as the final configuration.

Clone Score Calculation Parameters In order to determine the optimal combination of threshold θ and weight λ , we use the grid search method for systematic optimization. This method locates the best performing hyperparameter configuration by exhaustively evaluating a predefined parameter space. Based on preliminary analysis, we constructed a discrete search grid to explore the value range where each parameter is most likely to have an impact. The threshold θ is used to balance noise filtering and effective similarity signal retention. We defined the candidate set as 0.4, 0.5, and 0.6. The weight factor λ is used to balance the contribution of S_{MAS} and contextual support evidence S_{CSS} . The candidate values we selected are 0.75, 0.85, and 0.95. Thus, we constructed a 3x3 evaluation grid containing 9 sets of different parameter combinations. We experimented with all 9 combinations in the above space. Its MAP performance on the validation set is shown in Table 5

$\theta \setminus \lambda$	0.75	0.85	0.95
0.4	64.74	65.43	64.90
0.5	64.72	65.43	64.88
0.6	64.72	65.29	64.85

Table 5: Grid search results for θ and λ (MAP %)

From the results in Table 5, we observe that the model achieves its peak performance with a MAP score of 65.43% at two parameter combinations: when θ is 0.4 and λ is 0.85, and again when θ is 0.5 and λ is 0.85. Faced with this tie, we select $\theta=0.5$. This decision is based on the principle that a higher threshold establishes a more rigorous criterion for what constitutes a valid local match. The fact that performance does not degrade with this stricter standard indicates that the model’s predictive power is sustained by the most significant similarities and is not dependent on pairings with a lesser degree of resemblance. By selecting $\theta=0.5$, we favor a configuration that prioritizes more compelling evidence of similarity, which we posit leads to a more reliable and discerning clone detection model.

D. Performance Overhead Analysis

Our framework introduces computational overhead at several stages compared to simpler truncation-based methods. The overhead stems from three main sources: a pre-processing step to extract opcode sequences, an inference-time cost for encoding multiple overlapping blocks for each long code snippet, and a subsequent computational cost for constructing a complete $m \times n$ similarity matrix between all block pairs.

From a practical engineering standpoint, these costs are manageable. The opcode extraction and block encoding can be performed offline as a one-time pre-processing step, and the resulting block vectors can be cached. This leaves only the similarity matrix calculation and score aggregation for the online phase. While the matrix computation is not a trivial cost, it is a highly parallelizable and fast operation on modern hardware compared to deep learning inference, thus ensuring that our method remains practical for real-world deployment.

E. Code and Data Availability

To facilitate the reproduction of the experimental results of this paper, the complete source code, preprocessing scripts, and evaluation datasets are made publicly available at <https://github.com/lenny9776/LC3>.